

PROJETO DA DISCIPLINA

Neste trabalho, o aluno deverá desenvolver, em linguagem C, uma versão simplificada do famoso jogo Pac-Man. O objetivo é praticar conceitos fundamentais de programação, como leitura de arquivos, matrizes, estruturas de repetição, estruturas condicionais, funções, manipulação de caracteres e entrada de dados pelo teclado.

O jogo deverá utilizar um mapa carregado a partir de um arquivo .txt, que será impresso na tela e atualizado conforme os movimentos do jogador.

REPRESENTAÇÃO DOS ELEMENTOS DO MAPA

O mapa do jogo deverá ser armazenado em um arquivo de texto, utilizando os seguintes símbolos:

Símbolo Significado

.	Espaço vazio
	Parede vertical
–	Parede horizontal
@	Pac-Man
G	Fantasma
*	Ouro

Exemplo de mapa em arquivo .txt:

```
_____  
|.....|  
|..G....|  
|....*..|  
|...@...|  
|.....|  
_____
```

REGRAS BÁSICAS DO JOGO

O jogador controla o Pac-Man, representado pelo caractere @.

O Pac-Man deve se movimentar pelo mapa usando comandos do teclado, por exemplo:

Tecla	Movimento
W	Cima
S	Baixo
A	Esquerda
D	Direita

O Pac-Man **não pode atravessar paredes**, ou seja, não pode passar por posições ocupadas por | ou _.

Os fantasmas, representados por G, também **não podem atravessar paredes**.

O ouro, representado por *, deve ser coletado pelo Pac-Man. Ao passar por uma posição com ouro, o Pac-Man coleta o item e essa posição passa a ser ocupada por ele.

O mapa deve ser impresso novamente na tela a cada movimento realizado pelo jogador.

ATIVIDADES DO TRABALHO

1. CRIAR O ARQUIVO DE MAPA DO JOGO (2 PTS)

O aluno deverá criar um arquivo chamado, por exemplo:

mapa.txt

Nesse arquivo, deverá desenhar o mapa usando apenas os caracteres permitidos:

.
|
_
@
G
*

O mapa deve possuir apenas um Pac-Man @, pelo menos uma parede e pelo menos um ouro *.

2. DEFINIR AS DIMENSÕES MÁXIMAS DO MAPA

No código em C, o aluno deverá definir constantes para representar o tamanho máximo do mapa.

Exemplo:

```
#define LINHAS 20  
#define COLUNAS 40
```

Esses valores serão usados para criar uma matriz de caracteres que armazenará o mapa.

3. CRIAR UMA MATRIZ PARA ARMAZENAR O MAPA

O mapa lido do arquivo deverá ser armazenado em uma matriz de caracteres.

Exemplo:

```
char mapa[LINHAS][COLUNAS];
```

Cada posição da matriz representará uma posição do cenário do jogo.

4. LER O MAPA A PARTIR DO ARQUIVO .TXT

O programa deverá abrir o arquivo mapa.txt, ler seu conteúdo e armazenar cada caractere na matriz.

O aluno deverá utilizar funções como:

```
fopen()  
fgetc()  
fgets()  
fclose()
```

Também deverá verificar se o arquivo foi aberto corretamente.

5. IDENTIFICAR A POSIÇÃO INICIAL DO PAC-MAN

Após carregar o mapa, o programa deverá localizar onde está o caractere @.

Para isso, o aluno deverá percorrer a matriz procurando o Pac-Man.

Exemplo de variáveis:

```
int pacmanLinha;  
int pacmanColuna;
```

Essas variáveis serão usadas para controlar a posição atual do jogador.

6. IMPRIMIR O MAPA NA TELA

O aluno deverá criar uma função responsável por imprimir o mapa.

Exemplo:

```
void imprimirMapa(char mapa[LINHAS][COLUNAS]);
```

Essa função deverá percorrer a matriz e imprimir cada caractere na tela, mantendo o formato original do mapa.

A cada movimento do jogador, o mapa deverá ser impresso novamente.

7. CAPTURAR O MOVIMENTO DO JOGADOR

O programa deverá solicitar ao usuário que informe o movimento desejado.

Exemplo:

```
printf("Digite o movimento: ");  
scanf(" %c", &movimento);
```

Os comandos podem ser:

W - mover para cima
S - mover para baixo
A - mover para esquerda
D - mover para direita
Q - sair do jogo

O programa deverá aceitar letras maiúsculas ou minúsculas.

8. CALCULAR A NOVA POSIÇÃO DO PAC-MAN

Com base na tecla digitada, o programa deverá calcular a nova posição desejada.

Exemplo:

```
novaLinha = pacmanLinha;  
novaColuna = pacmanColuna;  
  
if (movimento == 'w' || movimento == 'W') {  
    novaLinha--;  
} else if (movimento == 's' || movimento == 'S') {  
    novaLinha++;  
} else if (movimento == 'a' || movimento == 'A') {  
    novaColuna--;  
} else if (movimento == 'd' || movimento == 'D') {  
    novaColuna++;  
}  
}
```

Neste momento, o programa ainda não move o Pac-Man. Ele apenas calcula para onde ele tentará ir.

9. VERIFICAR COLISÃO COM PAREDES

Antes de mover o Pac-Man, o programa deverá verificar se a nova posição contém uma parede.

O Pac-Man não pode se mover para posições ocupadas por:

|

–

Exemplo de lógica:

```
if (mapa[novaLinha][novaColuna] != '|' && mapa[novaLinha][novaColuna] != '_') {  
    // movimento permitido  
} else {  
    printf("Movimento inválido! O Pac-Man não pode atravessar paredes.\n");  
}
```

Se o movimento for permitido, a posição antiga do Pac-Man deverá virar espaço vazio . e a nova posição deverá receber @.

10. IMPLEMENTAR COLETA DE OURO, FANTASMAS E CONDIÇÃO DE TÉRMINO

O programa deverá verificar se o Pac-Man passou por uma posição com ouro *.

Quando isso acontecer, o jogador deverá ganhar pontos.

Exemplo:

```
int pontuacao = 0;
```

Ao coletar ouro:

```
pontuacao++;
```

Também deverá ser implementada uma regra para os fantasmas. Por exemplo:

Se o Pac-Man entrar em uma posição ocupada por G, o jogo termina.

Além disso, o trabalho poderá ter uma das seguintes condições de término:

- O jogador digita Q para sair;
- O Pac-Man coleta todos os ouros;
- O Pac-Man encosta em um fantasma.

Ao final, o programa deverá exibir a pontuação do jogador.

REQUISITOS MÍNIMOS DO TRABALHO (10 PTS)

O programa deverá obrigatoriamente:

1. Ler o mapa de um arquivo .txt;
2. Armazenar o mapa em uma matriz;
3. Imprimir o mapa na tela;
4. Localizar a posição inicial do Pac-Man;
5. Permitir movimentação com teclado;
6. Impedir que o Pac-Man atravesse paredes;
7. Permitir que fantasmas atravessem paredes;
8. Permitir coleta de ouro;
9. Atualizar o mapa após cada movimento;
10. Exibir uma mensagem de fim de jogo.